

Chapter 22. Statistics Without Fooling Yourself

Confidence intervals, tests, A/B design, and the traps that turn noise into findings.

Approved chapter v1

Contents

Chapter 22. Statistics Without Fooling Yourself

Why this matters

In plain English

22.1 Confidence intervals

22.2 Hypothesis tests and p-values

22.3 Designing an A/B test

22.4 Peeking, p-hacking, and multiple comparisons

22.5 Correlation, causation, and confounders

22.6 Simpson's paradox

22.7 Survivorship, selection, and regression to the mean

22.8 Statistical significance versus practical significance

22.9 Writing up a result

Common mistakes and how to spot them

In the real world: the transporter that wasn't better

Tools

The project: audit three claims

Timed challenge: forty minutes

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 22. Statistics Without Fooling Yourself

Part II — The Analyst

Chapter at a glance

You will learn to: turn sampling error into a confidence interval, for a mean and for a proportion, and explain what it does and doesn't mean · state a hypothesis, run the right test, and read a p-value without overclaiming · design an A/B test: what to measure, how many people you need, when to stop · see why peeking at a running test manufactures winners · handle multiple comparisons and recognize p-hacking in your own work · separate correlation from causation, and spot confounders, Simpson's paradox, survivorship bias, and regression to the mean · weigh statistical significance against practical significance · write up a result plainly, including the ones that didn't work.

Before you start: Chapter 21 (distributions, sampling, standard error, Bayes), Chapter 18 (pandas), and Chapter 15 (showing uncertainty). Chapter 4's percentages and Chapter 14's data quality both matter: no test survives bad data.

Time needed: 15–18 hours, spread over two weeks.

Tools: Python 3.13 or 3.14 with `pandas`, `numpy`, `scipy`, `matplotlib` (run here on Python 3.12, `pandas` 3.0.2, `numpy` 2.4.4, `scipy` 1.17.1). Excel can do most of it with `CONFIDENCE.NORM`, `T.TEST`, `CHISQ.TEST`, and the Analysis ToolPak.

Practice data: Chapter 21's `delivery_times_2025.csv`, plus `companion/ch22/ : ab_test_2026.csv` (an A/B test of two subject lines on Riverstone's February 2026 offer email, 8,400 recipients) and `transporters_q4_2025.csv` (11,600 Q4 deliveries by transporter and route). Both are invented for this chapter and generated by a seeded script; Riverstone's ERP has neither campaign nor transporter data.

Why this matters

Chapter 21 gave you the tools. This chapter is about the ways those tools go wrong in the hands of people who mean well, which includes you and me.

The pattern is always the same. Someone measures two things, sees a difference, and explains it. The explanation is plausible, the slide is clear, and the difference was noise. Or the difference was real and tiny, and the company spends ₹40 lakh chasing it. Or the difference was real and caused by something nobody put on the slide.

Analysts are *paid* to tell the difference. It's the one part of the job that a dashboard cannot do for you: deciding whether a number means anything. Get it right and you become the person whose numbers people trust when the room disagrees. Get it wrong twice in public and you are the person who

"made that chart".

Everything here is testable on Riverstone's data, and most of it is simulated rather than asserted: when the chapter says peeking at a test inflates false positives, it runs 3,000 tests and counts.

In plain English

You taste a spoonful of dal and decide the whole pot needs salt. That's statistics: a sample, a judgment about the pot, and a risk of being wrong.

Three things can go wrong, and they map onto the whole chapter.

- **The spoon was small.** One spoonful from a big pot might not be typical. That's sampling error, and a **confidence interval** is your way of saying "the pot is somewhere between this salty and that salty".
- **You stirred after tasting the first pot and not the second.** Now you're comparing a well-mixed pot with a settled one. That's a **confounder**: the thing you didn't control explains the difference you saw.
- **You kept tasting until it tasted how you hoped.** Taste twenty times and one spoonful will be unusual. That's **peeking and p-hacking**, and it's the most common way honest analysts produce false findings.

The discipline is not "be more careful". It's a small set of habits: decide what you'll measure before you look, quantify uncertainty every time, and ask what else could explain the number.

22.1 Confidence intervals

A sample mean is an estimate. A **confidence interval** is that estimate plus the range of values consistent with it.

```

import numpy as np
import pandas as pd
from scipy import stats
pd.set_option("display.width", 100)

deliveries = pd.read_csv("../ch21/delivery_times_2025.csv", parse_dates=["order_date"])
population = deliveries["order_value"]
rng = np.random.default_rng(22)

sample = pd.Series(rng.choice(population, size=200, replace=False))
mean = sample.mean()
standard_error = sample.std(ddof=1) / np.sqrt(len(sample))
lo, hi = stats.t.interval(0.95, df=len(sample)-1, loc=mean, scale=standard_error)

print(f"sample mean          ₹{mean:,.0f}")
print(f"standard error        ₹{standard_error:,.0f}")
print(f"95% interval           ₹{lo:,.0f} to ₹{hi:,.0f}")
print(f"the true mean          ₹{population.mean():,.0f} (inside the interval: {lo <=
population.mean() <= hi})")

```

```

sample mean          ₹23,280
standard error       ₹1,263
95% interval        ₹20,789 to ₹25,771
the true mean       ₹24,840 (inside the interval: True)

```

The recipe: **estimate $\pm$ (a multiplier $\times$ the standard error)**. The multiplier comes from the t-distribution (about 1.96 for 95% once the sample is large); the standard error is the sample standard deviation divided by $\sqrt{n}$ (Chapter 21, section 21.8).

What it means, and what it doesn't

Sixty samples of 200 orders: 56 intervals contain the truth, 4 miss it

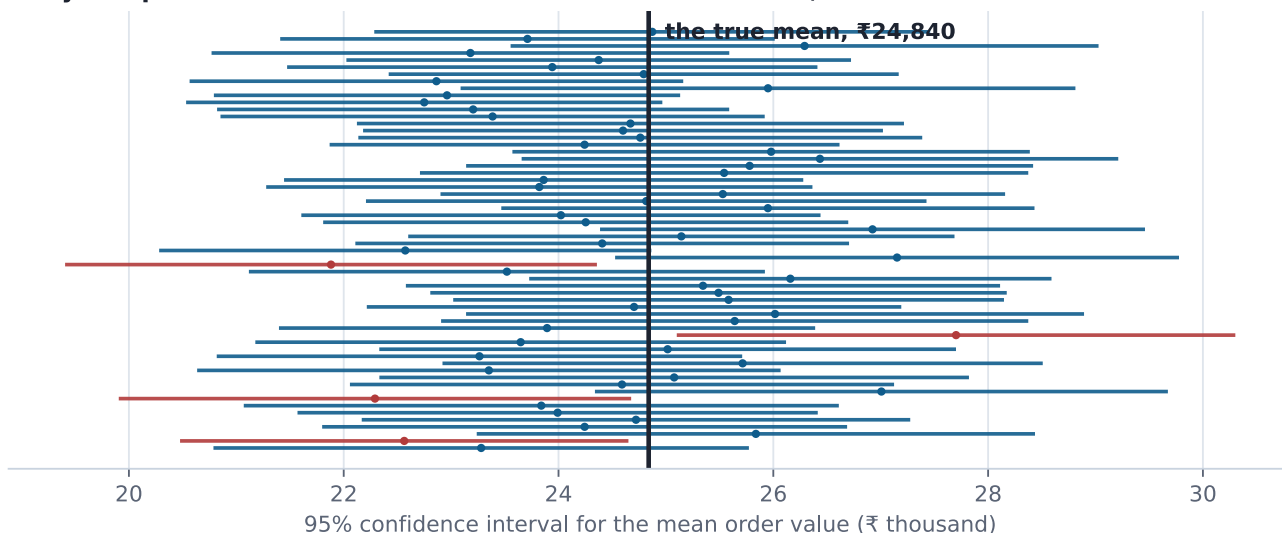


Figure 22.1 – Each line is one sample's 95% interval. The method is right 95% of the time; any single interval either contains the truth or doesn't.

- **Correct:** "If we repeated this sampling many times, 95% of the intervals we built this way would contain the true mean."
- **Wrong:** "There's a 95% probability the true mean is in *this* interval." The true mean is a fixed number; it's the interval that's random. (Bayesian statistics does let you make the first kind of

statement, with a different machinery and a prior; Chapter 21's Bayes' rule is the entry point.)

- **Also wrong:** "95% of orders fall in this range." That's a percentile range and it's much wider. Confidence intervals are about the *estimate*, not the data.

Intervals for proportions

Most business tests are about rates: open rate, conversion, on-time percentage.

```
on_time = deliveries["on_time"]
n, successes = len(on_time), int(on_time.sum())
p_hat = successes / n
se = np.sqrt(p_hat * (1 - p_hat) / n)
lo, hi = p_hat - 1.96*se, p_hat + 1.96*se
wilson_lo, wilson_hi = stats.binomtest(successes, n).proportion_ci(confidence_level=0.95,
method="wilson")

print(f"on-time rate {p_hat*100:.2f}% from {n:,} orders")
print(f"normal approximation: {lo*100:.2f}% to {hi*100:.2f}%")
print(f"Wilson interval:      {wilson_lo*100:.2f}% to {wilson_hi*100:.2f}%")

kolkata = deliveries[deliveries["branch"] == "Kolkata"]["on_time"]
k_lo, k_hi = stats.binomtest(int(kolkata.sum()), len(kolkata)).proportion_ci(method="wilson")
print(f"Kolkata: {kolkata.mean()*100:.2f}% ({k_lo*100:.2f}% to {k_hi*100:.2f}%) from
{len(kolkata):,} orders")
```

```
on-time rate 81.76% from 45,040 orders
normal approximation: 81.41% to 82.12%
Wilson interval:      81.40% to 82.12%
Kolkata: 68.15% (66.92% to 69.35%) from 5,626 orders
```

With 45,040 orders the interval is ± 0.4 points, so the on-time figure is precise. With a sample of 200 it would be ± 5 points, which is the difference between "we're at 82%" and "we're somewhere between 77% and 87%". **Report the interval whenever the number comes from a sample**, and especially when someone is about to compare it with a target.

Watch out: precision is not accuracy. A confidence interval measures sampling error only. It says nothing about a biased sample, a broken query, or the 100 customers with no city. Chapter 14's checks come first; the interval is the last step, not the first.

22.2 Hypothesis tests and p-values

A **hypothesis test** asks: if there were no real difference, how likely is a result at least as extreme as the one I saw?

- **The null hypothesis (H_0)** is the boring claim: the two subject lines have the same open rate; the branches have the same on-time rate.
- **The alternative (H_1)** is that they differ.
- **The p-value** is the probability of seeing your result, or one more extreme, *if the null were true*.
- **Significance level (α)**, usually 0.05, is the false-positive rate you accept before you look.

The A/B test

Riverstone tested two subject lines on its February 2026 offer email: 4,200 recipients each.

```
ab = pd.read_csv("ab_test_2026.csv")
summary = (ab.groupby("variant")
            .agg(sent=("opened", "size"), opens=("opened", "sum"),
                 orders=("ordered", "sum"), revenue=("order_value", "sum")))
summary["open_rate"] = (summary["opens"] / summary["sent"] * 100).round(2)
summary["order_rate"] = (summary["orders"] / summary["sent"] * 100).round(2)
summary["revenue_per_recipient"] = (summary["revenue"] / summary["sent"]).round(2)
print(summary)
```

variant	sent	opens	orders	...	open_rate	order_rate	revenue_per_recipient
A	4200	987	83	...	23.50	1.98	483.70
B	4200	1132	97	...	26.95	2.31	530.31

[2 rows x 7 columns]

Variant B was opened more often. Is that real?

```
from scipy.stats import chi2_contingency

opens = np.array([[int(summary.loc["A", "opens"]), int(summary.loc["A", "sent"] - summary.loc["A",
"opens"])],
                  [int(summary.loc["B", "opens"]), int(summary.loc["B", "sent"] - summary.loc["B",
"opens"])])])
chi2, p_open, dof, expected = chi2_contingency(opens, correction=False)
diff = summary.loc["B", "open_rate"] - summary.loc["A", "open_rate"]
se_diff = np.sqrt(sum((summary.loc[v, "opens"]/summary.loc[v, "sent"]) *
                      (1 - summary.loc[v, "opens"]/summary.loc[v, "sent"]) / summary.loc[v, "sent"])
for v in "AB"))
print(f"open rate: A {summary.loc['A', 'open_rate']}%, B {summary.loc['B', 'open_rate']}%, difference
+{diff:.2f} points")
print(f"95% interval for the difference: {diff/100 - 1.96*se_diff:+.4f} to {diff/100 +
1.96*se_diff:+.4f} (as a proportion)")
print(f"chi-square p-value: {p_open:.4f}")
```

open rate: A 23.5%, B 26.95%, difference +3.45 points
95% interval for the difference: +0.0159 to +0.0531 (as a proportion)
chi-square p-value: 0.0003

```
orders = np.array([[int(summary.loc["A", "orders"]), int(summary.loc["A", "sent"] -
summary.loc["A", "orders"])],
                   [int(summary.loc["B", "orders"]), int(summary.loc["B", "sent"] -
summary.loc["B", "orders"])])])
chi2_o, p_order, _, _ = chi2_contingency(orders, correction=False)
print(f"order rate: A {summary.loc['A', 'order_rate']}%, B {summary.loc['B', 'order_rate']}%")
print(f"chi-square p-value: {p_order:.4f}")
print(f"revenue per recipient: A ₹{summary.loc['A', 'revenue_per_recipient']:,.2f}, B ₹
{summary.loc['B', 'revenue_per_recipient']:,.2f}")
```

order rate: A 1.98%, B 2.31%
chi-square p-value: 0.2915
revenue per recipient: A ₹483.70, B ₹530.31

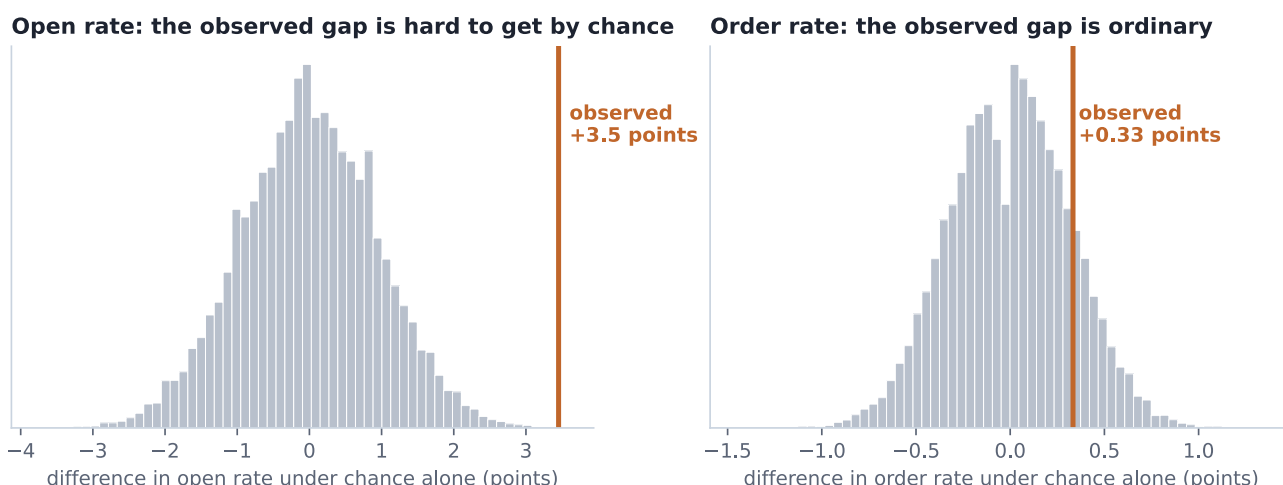


Figure 22.2 — What chance alone produces, and where the observed difference sits. Left: the open-rate gap is hard to explain by chance. Right: the order-rate gap is ordinary.

So: **B wins on opens and not on orders**. The honest conclusion is "the new subject line gets more people to open the email, and we have no evidence it sells more". Whether that's worth deploying depends on what you're optimizing, which is a business question, not a statistical one.

What a p-value is not

People say	It actually means
"p = 0.03, so there's a 3% chance the null is true"	No. It's the probability of the data given the null, not the null given the data (Chapter 21, section 21.7)
"p = 0.20, so there's no difference"	No. Absence of evidence isn't evidence of absence, especially in a small test
"p = 0.001, so the effect is big"	No. With enough data, a trivial difference gets a tiny p-value
"p = 0.049 significant, p = 0.051 not"	The threshold is a convention, not a law of nature. Report the number and the interval

Report the effect size and its interval first, the p-value second. "B's open rate is 3.45 points higher (95% interval: 1.6 to 5.3 points), p = 0.0003" tells a reader everything; "significant at p < 0.05" tells them almost nothing.

Choosing a test

Question	Test	In Python
Two proportions (open rate A vs B)	Chi-square, or a two-proportion z-test	<code>chi2_contingency</code> , <code>statsmodels.proportions_ztest</code>
Two means, roughly normal or large samples	Two-sample t-test	<code>stats.ttest_ind(a, b, equal_var=False)</code>
Two means, skewed data or small samples	Mann-Whitney U, or a bootstrap	<code>stats.mannwhitneyu</code> , or resampling
Before and after, same units	Paired t-test	<code>stats.ttest_rel</code>
More than two groups	ANOVA, then pairwise with a correction	<code>stats.f_oneway</code>

Question	Test	In Python
Categories against categories	Chi-square	chi2_contingency

```
mumbai = deliveries[deliveries["branch"] == "Mumbai H0"]["delivery_days"]
bengaluru = deliveries[deliveries["branch"] == "Bengaluru"]["delivery_days"]

t_stat, p_t = stats.ttest_ind(mumbai, bengaluru, equal_var=False)
u_stat, p_u = stats.mannwhitneyu(mumbai, bengaluru)
print(f"means: Mumbai {mumbai.mean():.2f} days, Bengaluru {bengaluru.mean():.2f} days")
print(f"Welch t-test p = {p_t:.3g}")
print(f"Mann-Whitney p = {p_u:.3g}")
print(f"difference in medians: {bengaluru.median() - mumbai.median():.1f} days")
```

```
means: Mumbai 3.51 days, Bengaluru 4.17 days
Welch t-test p = 1.93e-197
Mann-Whitney p = 0
difference in medians: 0.6 days
```

Both tests agree here, which is the usual outcome when the difference is real and the samples are large. The Mann-Whitney is the safer choice on skewed data because it compares ranks rather than means.

22.3 Designing an A/B test

Running the test is the simple part. The design is where it's won or lost.

1. Decide the metric before you look. One primary metric, chosen for the decision you'll make.

Riverstone's email test should have named *revenue per recipient* if the goal was sales, or *open rate* if the goal was reach. Choosing afterwards is how you always find a winner.

2. Work out the sample size you need. For a proportion, the standard formula needs the baseline rate, the smallest difference worth detecting (the **minimum detectable effect**), α (usually 0.05), and power (usually 0.80, the chance of detecting a real effect if it exists).

```
def sample_size_for_proportion(baseline, lift_points, alpha=0.05, power=0.80):
    """Recipients needed per variant to detect an absolute lift, in percentage points."""
    p1 = baseline
    p2 = baseline + lift_points / 100
    z_alpha = stats.norm.ppf(1 - alpha/2)
    z_beta = stats.norm.ppf(power)
    p_bar = (p1 + p2) / 2
    numerator = (z_alpha * np.sqrt(2 * p_bar * (1 - p_bar)) + z_beta * np.sqrt(p1*(1-p1) + p2*(1-
p2))) ** 2
    return int(np.ceil(numerator / (p2 - p1) ** 2))

for lift in (1, 2, 3, 5):
    print(f"to detect a {lift}-point lift on a 24% open rate: {sample_size_for_proportion(0.24,
lift):>7,} per variant")
for lift in (0.2, 0.5, 1.0):
    print(f"to detect a {lift}-point lift on a 2% order rate: {sample_size_for_proportion(0.02,
lift):>7,} per variant")
```

to detect a 1-point lift on a 24% open rate:	29,036 per variant
to detect a 2-point lift on a 24% open rate:	7,358 per variant
to detect a 3-point lift on a 24% open rate:	3,313 per variant
to detect a 5-point lift on a 24% open rate:	1,222 per variant
to detect a 0.2-point lift on a 2% order rate:	80,682 per variant
to detect a 0.5-point lift on a 2% order rate:	13,809 per variant
to detect a 1.0-point lift on a 2% order rate:	3,826 per variant

That table is the single most useful thing in this chapter for a working analyst. Riverstone's 4,200 per variant can detect about a 2.7-point difference in opens and nothing smaller than about 0.95 points in orders: fine for the first question, hopeless for the second. **Knowing this before the test means you either run it longer, or you don't pretend the order-rate result means anything.**

3. Randomize properly. Split at the level of the thing you're measuring, usually the customer, and make the split random. Splitting by branch, by day, or by "whoever was in the first half of the list" bakes in a confounder.

4. Run it for a whole number of business cycles. A week captures Monday behaviour and Saturday behaviour; three days captures whatever those three days were.

5. Decide the stopping rule in advance. This is the subject of the next section.

6. Write the plan down before starting. Metric, sample size, duration, stopping rule, and what decision each outcome leads to. A page is enough, and it's the difference between a test and a story.

22.4 Peeking, p-hacking, and multiple comparisons

Peeking

The tempting behaviour: check the dashboard every day, stop when it's significant. Here's what that does, with two **identical** variants and no real difference at all.

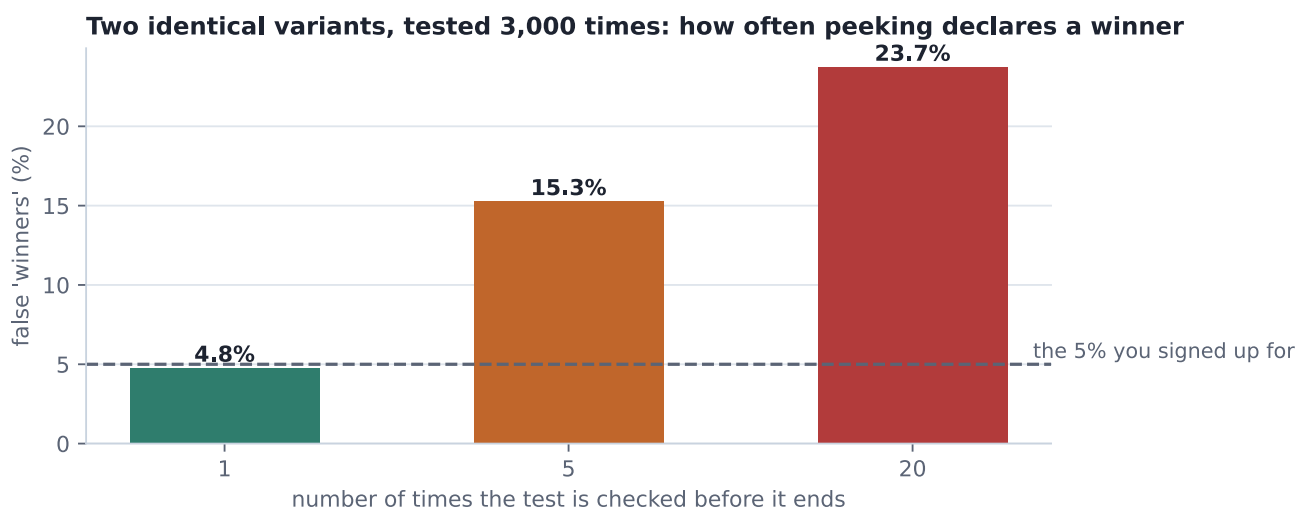


Figure 22.3 – Three thousand simulated tests of two identical variants. The more often you check, the more often chance hands you a "winner".

```

rng = np.random.default_rng(2223)

def peeking_false_positive_rate(peeks, trials=1500, n_max=4000, p=0.25):
    """Two identical variants. How often does checking `peek` times declare a winner?"""
    checkpoints = np.linspace(n_max / peaks, n_max, peaks).astype(int)
    false_positives = 0
    for _ in range(trials):
        a = rng.random(n_max) < p
        b = rng.random(n_max) < p
        for c in checkpoints:
            pa, pb = a[:c].mean(), b[:c].mean()
            se = np.sqrt(pa*(1-pa)/c + pb*(1-pb)/c)
            if se > 0 and abs(pa - pb) / se > 1.96:
                false_positives += 1
                break
    return false_positives / trials * 100

for peaks in (1, 5, 20):
    print(f"checking {peaks}>2} time(s): {peeking_false_positive_rate(peaks):.1f}% of identical
pairs declared a winner")

```

```

checking 1 time(s): 4.9% of identical pairs declared a winner
checking 5 time(s): 14.3% of identical pairs declared a winner
checking 20 time(s): 23.9% of identical pairs declared a winner

```

Checking once behaves as advertised: about 5% false positives. Checking twenty times nearly quintuples it. The fix is not "don't look" — it's **decide the sample size in advance and test once at the end**, or use a method designed for continuous monitoring (sequential testing, or a Bayesian approach with a stopping rule you set beforehand).

Multiple comparisons

The same arithmetic applies to testing many things at once. Test 20 independent hypotheses at $\alpha = 0.05$ and the chance of at least one false positive is $1 - 0.95^{20} = 64\%$.

```

for k in (1, 5, 10, 20, 50):
    print(f"{k}>2} independent tests at  $\alpha=0.05$ : P(at least one false positive) = {1 -
0.95**k:.1%}")

p_values = [0.001, 0.012, 0.030, 0.041, 0.048, 0.20, 0.31]
bonferroni = 0.05 / len(p_values)
print(f"\nBonferroni threshold for {len(p_values)} tests: {bonferroni:.4f}")
print("survive Bonferroni:", [p for p in p_values if p < bonferroni])

```

```

1 independent tests at  $\alpha=0.05$ : P(at least one false positive) = 5.0%
5 independent tests at  $\alpha=0.05$ : P(at least one false positive) = 22.6%
10 independent tests at  $\alpha=0.05$ : P(at least one false positive) = 40.1%
20 independent tests at  $\alpha=0.05$ : P(at least one false positive) = 64.2%
50 independent tests at  $\alpha=0.05$ : P(at least one false positive) = 92.3%

```

```

Bonferroni threshold for 7 tests: 0.0071
survive Bonferroni: [0.001]

```

Corrections: **Bonferroni** (divide α by the number of tests) is simple and strict; **Benjamini-Hochberg** controls the false discovery rate and is kinder when you're screening many things. In business analysis the practical version is: *if you sliced the data twenty ways and one slice was significant, treat it as a*

hypothesis for a new test, not as a finding.

p-hacking, without meaning to

The garden of forking paths is a series of small, reasonable decisions that each nudge the answer:

- Trying three metrics and reporting the one that worked.
- Removing outliers only when it helps.
- Adding a week because the result "hasn't settled".
- Splitting by segment after seeing the overall result was flat.
- Choosing a different test after the first one gave $p = 0.08$.

None of these feels dishonest. Together they turn a 5% false-positive rate into something much worse. The defences are boring and effective: **write the analysis plan first, report everything you tried, and keep confirmatory analysis separate from exploration**. Exploration is valuable; it produces hypotheses, not conclusions.

22.5 Correlation, causation, and confounders

Correlation measures whether two things move together, from -1 to +1.

```
by_customer = (deliveries.groupby("customer_id")
               .agg(orders=("order_id", "count"), revenue=("order_value", "sum"),
                    avg_days=("delivery_days", "mean")))
print(by_customer[["orders", "revenue", "avg_days"]].corr(numeric_only=True).round(3))
print()
print(f"Pearson (linear):    {stats.pearsonr(by_customer['orders'], by_customer['revenue'])
[0]:.3f}")
print(f"Spearman (rank):    {stats.spearmanr(by_customer['orders'], by_customer['revenue'])
[0]:.3f}")
```

	orders	revenue	avg_days
orders	1.000	0.915	-0.086
revenue	0.915	1.000	-0.071
avg_days	-0.086	-0.071	1.000

Pearson (linear):	0.915
Spearman (rank):	0.929

Orders and revenue correlate strongly, which is unsurprising and tells you nothing causal. The question that matters is always the same: **what else could produce this pattern?**

Four standard answers:

1. **Reverse causation.** Do late deliveries make customers order less, or do the customers who order oddly get worse service?
2. **A confounder.** A third thing causes both. Ice-cream sales and drownings rise together because of summer.
3. **Selection.** The data only includes cases where both things happened.

4. **Chance**, especially with many variables (section 22.4).

Pearson measures linear association; **Spearman** measures monotonic association on ranks and stands up to outliers and skew, which makes it the safer default for business data. And a correlation near zero means no *linear* relationship, not no relationship: a U-shape can have $r \approx 0$.

22.6 Simpson's paradox

Riverstone's operations team compared its two transporters on Q4 deliveries.

```
transporters = pd.read_csv("transporters_q4_2025.csv")
overall = transporters.groupby("transporter")["on_time"].agg(["size", "mean"])
overall["on_time_pct"] = (overall["mean"] * 100).round(1)
print(overall[["size", "on_time_pct"]])
```

	size	on_time_pct
transporter		
BlueCart	5500	79.7
SwiftLine	6100	91.1

SwiftLine is 11 points better, and the obvious conclusion is to give SwiftLine more work. Now split by route type:

```
by_route = (transporters.groupby(["transporter", "route"])["on_time"]
            .agg(deliveries="size", on_time_pct=lambda s: round(s.mean()*100, 1))
            .reset_index())
print(by_route.to_string(index=False))
print()
mix = (transporters.groupby(["transporter", "route"]).size() /
      transporters.groupby("transporter").size() * 100).round(1)
print("route mix (% of each transporter's deliveries):")
print(mix.unstack().to_string())
```

transporter	route	deliveries	on_time_pct
BlueCart	Metro	1100	95.3
BlueCart	Upcountry	4400	75.8
SwiftLine	Metro	5200	94.1
SwiftLine	Upcountry	900	74.2

```
route mix (% of each transporter's deliveries):
route      Metro  Upcountry
transporter
BlueCart    20.0    80.0
SwiftLine   85.2    14.8
```

BlueCart is better on both routes and worse overall. That's **Simpson's paradox**, and it isn't a trick: SwiftLine does 85% of its work on the simpler metro routes, BlueCart does 80% on hard upcountry ones. The overall figure mostly measures the mix of work, not the quality of the transporter.

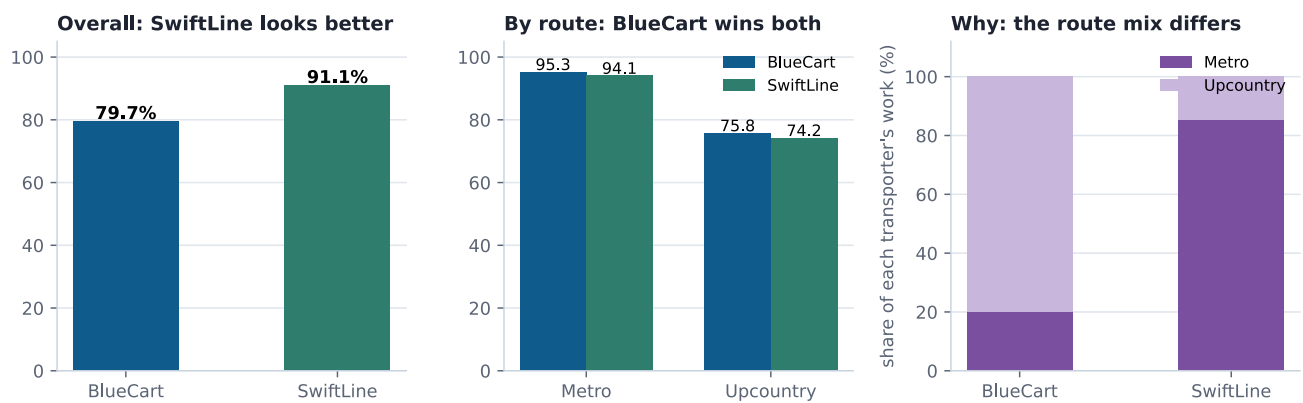


Figure 22.4 – The same data, three views. The third panel is the explanation: the transporters aren't doing the same job.

The fix is to compare like with like:

```
weights = transporters.groupby("route").size() / len(transporters)
adjusted = (by_route.pivot(index="transporter", columns="route", values="on_time_pct")
             .mul(weights * 100 / 100, axis=1).sum(axis=1).round(1))
print("on-time rate if both transporters had the company's route mix:")
print(adjusted.to_string())
```

```
on-time rate if both transporters had the company's route mix:
transporter
BlueCart      86.4
SwiftLine     85.0
```

That's **standardization**: recompute each transporter's rate as if it handled the company's overall mix of routes. On a level footing, BlueCart is ahead.

Where this shows up in ordinary work: branch comparisons where the customer mix differs, year-on-year margins when the product mix shifts, hospital outcomes when one hospital takes the difficult cases, and every "our conversion fell" that turns out to be a change in traffic sources. **Whenever you compare groups, ask what differs between them besides the label.**

22.7 Survivorship, selection, and regression to the mean

Survivorship bias

The data you have is the data that survived. Riverstone's `customers` table contains the customers who signed up and stayed; the ones who left after one bad delivery are still there but silent, and the ones who never signed up are invisible. Analyze "our customers love the new product" from a survey of current customers and you've measured the people who didn't leave.

The classic version is the wartime study of returning aircraft: reinforcing where the bullet holes were would have been backwards, because the planes hit elsewhere never came back. The business version is "all our best customers use feature X", where the customers who tried X and hated it are not in the table.

Selection bias

```
survey_responders = deliveries[deliveries["delivery_days"] >
deliveries["delivery_days"].quantile(0.75)]
print(f"all orders:          mean {deliveries['delivery_days'].mean():.2f} days, on time
{deliveries['on_time'].mean()*100:.1f}%")
print(f"if only the slowest quarter answered the survey: mean
{survey_responders['delivery_days'].mean():.2f} days, "
      f"on time {survey_responders['on_time'].mean()*100:.1f}%")
```

```
all orders:          mean 4.37 days, on time 81.8%
if only the slowest quarter answered the survey: mean 7.29 days, on time 28.0%
```

If the people most likely to answer a delivery survey are the ones who waited longest, the survey measures their experience, not the company's. Response rates matter more than sample size: 3,000 responses from 2% of customers can be more biased than 300 from a random sample of 30%.

Regression to the mean

Extreme results tend to be followed by less extreme ones, with no intervention at all.

```
rng = np.random.default_rng(2207)
customers = by_customer[by_customer["orders"] >= 8].copy()
first_half = rng.normal(0, 1, len(customers))          # a measure with no real trend
second_half = rng.normal(0, 1, len(customers))
worst = np.argsort(first_half)[:50]

print(f"the 50 'worst' in period 1: mean {first_half[worst].mean():+.2f} standard deviations")
print(f"the same 50 in period 2:      mean {second_half[worst].mean():+.2f} standard deviations")
print("nothing was done to them between the periods")
```

```
the 50 'worst' in period 1: mean -2.29 standard deviations
the same 50 in period 2:      mean -0.13 standard deviations
nothing was done to them between the periods
```

Pick the worst 50 branches, reps, or customers on a noisy measure, do *anything*, and they will improve. That improvement is not evidence the intervention worked; it's arithmetic. The defences: a **control group** that also had a bad period, or a comparison of the change against the change in everyone else.

This is why "we retrained the bottom 10% of reps and their performance rose 18%" is one of the least informative sentences in business reporting, and why the version with a control group is one of the most.

22.8 Statistical significance versus practical significance

```
big_n = 500_000
rate_a, rate_b = 0.2400, 0.2436          # a 0.36-point difference
successes = np.array([[int(rate_a*big_n), int((1-rate_a)*big_n)],
                      [int(rate_b*big_n), int((1-rate_b)*big_n)]])
chi2_big, p_big, _, _ = chi2_contingency(successes, correction=False)
print(f"with {big_n:,} per variant, a {(rate_b-rate_a)*100:.2f}-point difference gives p = {p_big:.2e}")

extra_opens = (rate_b - rate_a) * big_n
print(f"that is {extra_opens:,.0f} extra opens per {big_n:,} emails")
print(f"at Riverstone's real list size of 8,400, it would be {(rate_b-rate_a)*8400:.0f} extra opens")
```

```
with 500,000 per variant, a 0.36-point difference gives p = 2.62e-05
that is 1,800 extra opens per 500,000 emails
at Riverstone's real list size of 8,400, it would be 30 extra opens
```

A large enough sample makes any difference "significant". The questions that decide whether it matters:

- **How big is the effect, in units the business cares about?** Rupees, orders, days, complaints.
- **What does acting on it cost?** A subject-line change costs nothing; a new transporter costs a contract.
- **Would the decision change if the effect were at the pessimistic end of the interval?** If not, you don't need more data.
- **Is it stable?** One quarter's effect that vanishes next quarter wasn't an effect.

The honest write-up has four parts: the **effect**, its **interval**, the **decision it supports**, and the **assumptions that could overturn it**.

Watch out: the difference of two significances is not a significance. "Sales rose significantly in the North and not in the South" does not mean the regions differ. To claim that, test the difference between them directly. This mistake is everywhere, including in published research.

22.9 Writing up a result

A template that survives scrutiny:

What we tested. Two subject lines for the February offer email, randomly assigned to 4,200 recipients each, measured for seven days. Primary metric: revenue per recipient, decided before the test.

What we found. Variant B was opened 3.45 points more often (23.50% against 26.95%, 95% interval 1.6 to 5.3 points, $p = 0.0003$). Order rate and revenue per recipient were not distinguishable (order rate 1.98% against 2.31%, $p = 0.29$; revenue per recipient ₹483.70 against ₹530.31).

What it means. The new subject line reaches more people; we have no evidence it sells more. With 4,200 per variant we could only have detected an order-rate difference of about 0.95 points, so "no difference" here means "no large difference".

What we recommend. Adopt B for reach, at no cost. If order rate is the goal, run a test of at least 25,000 per variant, or test the offer rather than the subject line.

What could overturn this. The test ran in one week of February, on the full list; behaviour may differ in the festive season, and we didn't test on the lapsed segment.

Five habits behind it:

1. **Lead with the effect, not the p-value.**
2. **Always give the interval**, and say what you couldn't have detected.
3. **Report the metrics you didn't use**, and say they weren't primary.
4. **State the population and period**, because that's what the result applies to.
5. **Say what would change your mind.** It's the fastest way to be trusted, and it makes the next test easier to justify.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Quoting a sample number with no interval	A 2-point "improvement" from 200 responses	Report the confidence interval every time
Reading an interval as a probability about the truth	"95% chance the mean is in here"	The method is 95% right; this interval either contains it or not
Confusing a confidence interval with a data range	"95% of orders are in this range"	Percentiles describe data; intervals describe estimates
Treating p as the probability the null is true	"3% chance there's no effect"	p is the probability of the data given the null
Treating $p > 0.05$ as proof of no effect	"No difference" from an underpowered test	State the minimum detectable effect
Choosing the metric after the result	Every test has a winner	Name the primary metric in advance
Peeking until significant	20–25% false winners	Fix the sample size; test once at the end

Mistake	Symptom	Fix
Testing twenty slices and reporting one	A "finding" in exactly one segment	Correct for multiple comparisons; retest it
Removing outliers only when it helps	Results that move with the analyst's mood	Decide the rule first; report both ways
Randomizing by branch or day	The confounder is in the split	Randomize at the unit you measure
Running a test for three days	Day-of-week effects masquerade as lift	Whole business cycles
Reading correlation as causation	"Late deliveries cause churn"	Ask about reverse causation, confounders, selection
Comparing groups with different mixes	Simpson's paradox	Split by the confounder; standardize
Analysing only the survivors	"Our customers are all happy"	Ask who is missing from the table
Acting on the worst performers' rebound	"Coaching improved the bottom 10% by 18%"	Control group, or compare changes
Confusing significance with importance	$p = 1e-12$ on a 0.04-point lift	Effect size in business units
Comparing two significances	"Significant in North, not in South"	Test the difference between groups
Hiding the tests that failed	A suspiciously tidy story	Report what you tried

In the real world: the transporter that wasn't better

In January 2026, Riverstone's logistics head proposed moving most upcountry deliveries to SwiftLine. The case was one number: 91.1% on-time against BlueCart's 79.7% across 11,600 Q4 deliveries. With that many deliveries, nobody doubted the number, and nobody needed to: it was right.

Meera asked one question before the contract went to procurement: *do the two transporters carry the same kind of work?*

They did not. SwiftLine ran 5,200 metro deliveries and 900 upcountry ones; BlueCart ran 1,100 metro and 4,400 upcountry. Split by route, the ranking flipped: BlueCart was better on metro (95.3% against 94.1%) and better upcountry (75.8% against 74.2%). Standardized to the company's actual route mix, BlueCart came out ahead overall.

The rest of the meeting was about two things, and both were more useful than the original proposal:

1. **Upcountry is the problem, not the transporter.** Both carriers lose about a quarter of upcountry deliveries against promise. Moving work between them changes the headline and not the customer experience.
2. **The headline number measures the mix.** If SwiftLine took the upcountry volume, its overall rate would fall toward BlueCart's, and the following quarter's review would "discover" that SwiftLine had deteriorated.

Riverstone kept both transporters, set route-level targets instead of company-level ones, and started reporting on-time rates standardized to a fixed route mix so that quarter-to-quarter comparisons meant something.

A postscript worth keeping. Three months later the upcountry rate improved by four points after a routing change. The logistics head asked whether it was real. It was: the improvement held in both transporters, in every month, and the 95% interval on the change (± 1.1 points) excluded zero comfortably. That's the same discipline pointed the other way — being able to say "this one is real" is exactly what makes the earlier "this one isn't" credible.

What made the difference:

- **One question:** are we comparing like with like?
- **Splitting by the obvious confounder** before accepting a difference.
- **Standardizing** so the comparison could be repeated fairly.
- **Being equally rigorous about the good news**, which is how you keep the right to question the bad.

Tools

- **Python 3.13 or 3.14** with `pandas`, `numpy`, `scipy`, `matplotlib`. Run here on Python 3.12 with `pandas` 3.0.2, `numpy` 2.4.4, `scipy` 1.17.1. `statsmodels` adds `proportions_ztest`, `power` calculators, and multiple-comparison corrections, and is worth installing.
- **Excel:** `CONFIDENCE.NORM`, `CONFIDENCE.T`, `T.TEST`, `CHISQ.TEST`, `NORM.S.INV`, and the Analysis ToolPak's t-Test, ANOVA, and Descriptive Statistics.
- **Calculators** for sample size and A/B significance are everywhere online; use them for a second opinion, not as the method of record, and check what they assume about one- versus two-sided tests.
- **Companion files (companion/ch22/):** `build_ch22_files.py` (seed 202201), `ab_test_2026.csv` (8,400 recipients across two variants), `transporters_q4_2025.csv` (11,600 deliveries by transporter and route, built to contain a genuine Simpson's paradox). Both datasets are invented for this chapter; the delivery data comes from Chapter 21.

The project: audit three claims

Goal: take three claims that would pass unchallenged in a meeting, and test each one properly.

Option A: your own workplace. Three recent claims from a report, a review, or a vendor.

Option B: Riverstone. Use these three:

1. "Variant B of the offer email is the better subject line."
2. "SwiftLine is our better transporter."

3. **"Kolkata's on-time performance improved this quarter"** (use the festive months against the rest of 2025 as a stand-in).

Steps, for each claim

1. **Write the claim as a testable statement**, with a metric, a population, and a period.
2. **Check the data first** (Chapter 14): missing rows, duplicates, the period, and what's excluded.
3. **Estimate the effect and its confidence interval**, before any test.
4. **Run the appropriate test** and report the p-value alongside the effect.
5. **List the confounders** and split by at least one of them. Standardize if the mixes differ.
6. **Say what you could have detected**: compute the minimum detectable effect for the sample you have.
7. **Write the verdict in the section 22.9 template**, including what would change your mind.
8. **Recommend a decision**, and say what it would cost to be wrong.

What good looks like: claim 1 survives on opens (+3.45 points, 95% interval 1.6 to 5.3, $p = 0.0003$) and fails on orders ($p = 0.29$, minimum detectable effect about 0.95 points). Claim 2 reverses once route is controlled for. Claim 3 gets the honest treatment of a comparison that mixes seasonal effects with everything else.

Stretch goals

- Bootstrap the difference in median delivery days between two branches (resample 10,000 times) and compare with the t-test's interval.
- Design the order-rate test properly: how many recipients, over how long, and what you'd do at each outcome.
- Simulate your own peeking experiment with a real effect present, and measure how often early stopping overstates the effect size.
- Take any monthly KPI in your workplace, compute its month-to-month standard deviation, and work out how large a move has to be before it means anything.

Timed challenge: forty minutes

Use `ab_test_2026.csv`, `transporters_q4_2025.csv`, and Chapter 21's delivery data. Answers at the end.

- **Level 1:** Open rate and order rate for each variant, with the difference in points.
- **Level 2:** A 95% confidence interval for each variant's open rate.
- **Level 3:** The p-value for the open-rate difference, and for the order-rate difference.
- **Level 4:** The minimum detectable effect for the order rate at 4,200 per variant.
- **Level 5:** On-time rate by transporter, overall and by route.
- **Level 6:** Each transporter's on-time rate standardized to the company's route mix.
- **Level 7:** A 95% interval for Kolkata's on-time rate, and for Mumbai HO's. Do they overlap?

- **Bonus:** Simulate 1,000 A/A tests on the email data (split variant A randomly in two) and count how often $p < 0.05$.
-

You've got it when...

- ☐ You quote an interval with every estimate from a sample, and can say what it means without overclaiming.
 - ☐ You can state a null hypothesis and choose the right test for two proportions, two means, and skewed data.
 - ☐ You lead with effect size and interval, and treat p as supporting evidence.
 - ☐ You compute the sample size before running a test, and the minimum detectable effect after.
 - ☐ You fix the stopping rule in advance and can explain why peeking inflates false positives.
 - ☐ You correct, or at least disclose, when you've tested many things.
 - ☐ You ask what else could explain a correlation, and name the likely confounders.
 - ☐ You check whether a group comparison is comparing like with like, and standardize when it isn't.
 - ☐ You can spot survivorship, selection, and regression to the mean in someone else's chart.
 - ☐ You can say whether an effect matters to the business, not only whether it's significant.
 - ☐ You write results with the effect, the interval, the decision, and what would change your mind.
-

Recap

- **Confidence intervals** turn sampling error into a range. The method is right 95% of the time; a particular interval either contains the truth or doesn't. Use Wilson intervals for proportions.
- **A p-value** is the probability of the data given the null, never the probability of the null. Report the effect and its interval first.
- **Choose the test by the data:** chi-square for proportions, Welch's t-test for means, Mann-Whitney for skewed data, paired tests for before-and-after.
- **Design before you run:** one primary metric, a sample size from a power calculation, randomization at the unit you measure, whole business cycles, and a stopping rule.
- **Peeking manufactures winners:** 5% false positives becomes 15% with five looks and 24% with twenty, in simulation on identical variants.
- **Multiple comparisons:** twenty tests at $\alpha = 0.05$ give a 64% chance of at least one false positive. Correct, or treat the finding as a hypothesis.
- **Correlation isn't causation:** consider reverse causation, confounders, selection, and chance. Spearman is the safer correlation on business data.
- **Simpson's paradox** is common wherever groups do different work: BlueCart is better on both routes and worse overall, because it carries the hard routes. Standardize to compare fairly.

- **Survivorship and selection** decide who is in your data; **regression to the mean** explains most "the worst performers improved" stories.
- **Significance is not importance.** With half a million rows, a 0.36-point difference has $p \approx 10^{-8}$ and means almost nothing.
- **Write it up plainly:** effect, interval, what you couldn't detect, the decision, and what would change your mind.

Practice exercises

Warm-up

1. Rewrite each without overclaiming: (a) "There's a 95% chance the true on-time rate is between 81.4% and 82.2%." (b) " $p = 0.30$, so the two subject lines are the same." (c) " $p = 0.001$, so the effect is large."
2. A survey of 400 customers finds 62% satisfied. What's the 95% interval, and how would you present the number to a manager?
3. Why is a confidence interval not the same as the range that 95% of orders fall in?
4. You run a test for four days and see a 6% lift with $p = 0.04$. Give three reasons not to announce it yet.
5. Name three confounders that could explain "customers who receive our newsletter order more".
6. What is the minimum detectable effect, and why does it matter more than the p-value in a test that found nothing?

Core

7. Compute open rate, order rate, and revenue per recipient for each variant, with a 95% interval for each rate.
8. Test the open-rate difference with a chi-square test and with a two-proportion z-test. Do they agree?
9. Compute the 95% interval for the *difference* in open rates. Does it exclude zero?
10. Test the order-rate difference. What's the p-value, and what's the largest effect you could have ruled out?
11. How many recipients per variant would you need to detect a 0.5-point lift on a 2% order rate, at 80% power?
12. Test whether Mumbai HO and Bengaluru differ in mean delivery days (Welch) and in distribution (Mann-Whitney). Report the effect in days.
13. Compute the on-time rate and its 95% interval for each branch. Which pairs of branches have non-overlapping intervals?
14. Reproduce the Simpson's paradox table, then standardize both transporters to the company route mix. State the conclusion in one sentence.
15. Simulate the peeking experiment with 1, 3, 10, and 30 checkpoints. Plot the false-positive rate.

16. Run 1,000 A/A tests by randomly splitting variant A's recipients in two. How often is $p < 0.05$, and is that what you expected?
17. Compute the correlation between orders and average delivery days per customer, with Pearson and Spearman. What would you need to claim causation?
18. Split the delivery data into festive (October–November) and the rest. Is Kolkata's on-time rate different, and what confounds the comparison?
19. Bootstrap the difference in median delivery days between Delhi and Kolkata (10,000 resamples) and report a 95% interval.
20. Demonstrate regression to the mean: take the 100 customers with the lowest average delivery time in the first half of 2025, and report their average in the second half.
21. Write up the A/B test in the section 22.9 template, in under 200 words.
22. Take a claim from your own workplace and test it with steps 1–8 of the project.

Stretch

23. Implement a Bonferroni and a Benjamini-Hochberg correction for a list of 12 p-values, and compare which survive.
24. Compute the power of the order-rate test as actually run (4,200 per variant, observed rates). What does that say about its conclusion?
25. Build a simple sequential test: check every 500 recipients but require $p < 0.005$ to stop early. Simulate its false-positive rate against the naive version.
26. Using only the delivery data, construct your own Simpson's paradox by choosing a grouping and a confounder. Then explain why it happens.

Think about it

27. A vendor says their tool "improved conversion by 14% for our clients". What do you ask?
28. Your A/B test shows a 0.4-point lift with $p = 0.19$. The sponsor wants to ship it because "it's positive". What do you say?
29. Is it ever right to act on a result that isn't statistically significant?

Key terms

sampling error · confidence interval · confidence level · margin of error · standard error · t-distribution · Wilson interval · null hypothesis · alternative hypothesis · p-value · significance level (α) · type I and type II error · power · minimum detectable effect · chi-square test · two-proportion z-test · Welch's t-test · Mann-Whitney U · paired test · ANOVA · effect size · A/B test · randomization unit · primary metric · stopping rule · peeking · sequential testing · multiple comparisons · family-wise error rate · Bonferroni correction · Benjamini-Hochberg · false discovery rate · p-hacking · garden of forking paths · pre-registration · correlation · Pearson · Spearman · causation · reverse causation · confounder · Simpson's paradox · standardization · survivorship bias · selection bias · non-response bias · regression to the mean · control group · practical significance · bootstrap

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 23, Data Storytelling:** how to present an uncertain result so that it's understood rather than smoothed away.
 - **Chapter 24, Forecasting:** prediction intervals, and the same humility applied to the future.
 - **Chapter 25:** experimentation in practice, where the constraints are organizational as much as statistical.
 - **Chapter 27:** the ethics of analysis, including selective reporting.
 - **Part IV:** train/test splits, cross-validation, and overfitting are this chapter's ideas in a modelling coat.
 - **Interview preparation:** the Statistics & Analytics Question Bank (Chapter 73) covers p-values, A/B design, and "why might this result be wrong?", which is the most common senior-analyst interview question there is.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) "If we repeated this sampling many times, 95% of the intervals built this way would contain the true rate; this one runs from 81.4% to 82.2%." (b) "We found no evidence of a difference; with this sample we could only have detected a difference of about 0.95 points or more." (c) "The difference is unlikely to be chance; it is 0.36 points, which is small."
2. $62\% \pm 1.96 \times \sqrt{(0.62 \times 0.38 \div 400)} = \mathbf{57.2\% \text{ to } 66.8\%}$. Present it as "about 62%, give or take five points", and say the sample size.
3. The interval describes uncertainty about an *estimate* and shrinks with more data; the 95% range of the data describes the spread of orders and doesn't shrink at all. On Riverstone's order values the interval for the mean from 200 orders is a few thousand rupees wide; the range holding 95% of orders is tens of thousands.
4. Four days isn't a whole business cycle; $p = 0.04$ after repeated checking isn't 0.04; and a 6% lift early in a test is usually an overstatement, because tests that stop early stop on favourable noise.
5. Newsletter subscribers are already more engaged; they're often existing large customers; and they may have opted in at the point of a purchase. Each explains the association without the newsletter causing anything.
6. The smallest true effect your test had a good chance of detecting. In a null result it's the whole story: "no difference" with a minimum detectable effect of 5 points means you learned very little.

7. A: opens 23.50% (95% interval about 22.2% to 24.8%), orders 1.98%, revenue per recipient ₹483.70. B: opens 26.95% (25.6% to 28.3%), orders 2.31%, ₹530.31.
8. Both give $p \approx 0.0003$; the chi-square statistic is the square of the z statistic for a 2×2 table, so they always agree.
9. The difference is +3.45 points, 95% interval about **+1.6 to +5.3 points**. It excludes zero, which is the same conclusion as the p-value in a more useful form.
10. $p = 0.29$. At 4,200 per variant and a 2% baseline, the minimum detectable effect at 80% power is about **0.95 points**, so any true difference smaller than that was likely to be missed.
11. About **13,800** per variant.
12. Mumbai HO's mean is 3.51 days against Bengaluru's 4.17, a difference of 0.66 days; both tests give p far below 0.001. The medians are 3.2 and 3.8, so 0.6 days is the number to quote.
13. Mumbai HO 90.8% (90.3–91.2), Delhi 80.0% (79.3–80.8), Bengaluru 78.0% (77.3–78.7), Kolkata 68.1% (66.9–69.4). Every pair is non-overlapping except Delhi and Bengaluru, which are close but still separated at these sample sizes.
14. Overall SwiftLine 91.1%, BlueCart 79.7%; by route BlueCart wins both (95.3 vs 94.1 metro, 75.8 vs 74.2 upcountry); standardized to the company mix the order reverses. One sentence: *"BlueCart performs better on both route types; SwiftLine's higher overall rate reflects an easier route mix, not better service."*
15. The false-positive rate rises steeply and then flattens: roughly 5%, 11%, 19%, and 27% for 1, 3, 10, and 30 checkpoints in this setup. The exact numbers depend on the simulation, but the shape does not.
16. About **5%** of A/A tests come out "significant", which is exactly what $\alpha = 0.05$ promises. Running this once is the fastest cure for trusting a single significant result.
17. The two correlate weakly at best; Spearman and Pearson will differ because both variables are skewed. To claim causation you would need an experiment (randomly assigning delivery speed) or a natural one, plus control for branch, segment, and order size.
18. Kolkata's on-time rate is markedly lower in October and November. The comparison is confounded by volume (the festive peak), route mix, and the transporter mix, so a raw before-and-after overstates any process change.
19. Resample each branch's delivery days 10,000 times, take the difference of medians each time, and report the 2.5th and 97.5th percentiles. The interval will be comfortably above zero, around 1.2 to 1.4 days.
20. Their average in the second half is closer to the company average than their first-half figure: the extremes regress. That's the null result you must be able to produce before believing any "we fixed the worst performers" claim.
21. See the template in section 22.9; the numbers are in answers 7 to 10.
22. The point of the exercise is the write-up, not the test: a claim, a metric, an interval, a confounder you checked, and a decision.

23. With Bonferroni at 12 tests, only $p < 0.0042$ survives; Benjamini-Hochberg keeps more of the borderline ones by controlling the false discovery rate instead of the family-wise error rate. Use Bonferroni when a false positive is expensive, BH when you're screening.
24. Power for the observed 0.33-point difference at 4,200 per variant is low (well under 30%), which is why "no significant difference" is uninformative here: the test was never likely to detect an effect of that size.
25. The sequential version keeps the overall false-positive rate near 5% while allowing early stopping, at the cost of needing a larger effect to stop early. That trade — stricter threshold for the right to look — is what all sequential methods buy.
26. Any grouping where the mix differs works: branch and order size, month and segment, or customer type and product. It happens because the group with the harder mix is penalized by the overall average, and the fix is always the same: compare within strata, or standardize.
27. From what baseline, over what period, on what population? Was it a controlled test or a before-and-after? How many clients, and were the ones who didn't improve included? Is 14% relative or absolute? And what's the confidence interval? "14% relative on a 1.2% baseline, uncontrolled, best five clients" is a different claim from the one being made.
28. Say what the data supports: the observed lift is 0.4 points, the interval includes zero and runs from about -0.2 to +1.0, so the result is consistent with anything from a small loss to a small gain. Then make it a business decision: if shipping costs nothing and the downside is bounded, ship it and keep measuring; if it costs real money, run the test to the sample size that could answer the question.
29. Yes, when the cost of acting is low and the cost of waiting is high, when the direction is consistent with other evidence, or when the decision has to be made anyway. What's not legitimate is *reporting* it as a proven effect. Say "consistent with a small improvement, not yet established", act if the economics justify it, and keep measuring.
- Timed challenge answers.** Level 1: A 23.50% opens and 1.98% orders; B 26.95% and 2.31%; differences +3.45 and +0.33 points. Level 2: A about 22.2–24.8%, B about 25.6–28.3%. Level 3: opens $p = 0.0003$, orders $p = 0.29$. Level 4: about 0.95 points. Level 5: SwiftLine 91.1% overall (94.1% metro, 74.2% upcountry), BlueCart 79.7% overall (95.3% metro, 75.8% upcountry). Level 6: standardized to the company mix, BlueCart is ahead of SwiftLine. Level 7: Kolkata 66.9–69.4%, Mumbai HO 90.3–91.2%: nowhere near overlapping. Bonus: about 5% of the A/A tests come out significant.